

Universal Save — Product Plan

(working title — naming ideas at the bottom)

1. One-liner

A place to save anything you find online — link, tweet, article, PDF, video — that quietly organizes itself and taps you on the shoulder when it's relevant again.

2. The Problem

People save things constantly (tabs, tweets, screenshots, "read later" links) but almost never go back to them. Bookmarks become a graveyard. The failure isn't *saving* — it's **retrieval**: nothing resurfaces the item at the moment it would actually be useful, and nothing connects it to the other things you've saved on the same topic.

3. Target User

You want this open to anyone — that's the right long-term vision, and the "paste any link" mechanic supports it. But "anyone" isn't a marketing wedge; nobody searches for "app for anyone." For launch, pick one beachhead group to write copy for and recruit first users from, while the product itself stays general-purpose. Good candidates, since they all save heavily and already pay for adjacent tools: students/researchers, content creators, or generalist "online knowledge worker" types. You don't have to decide this today — but decide it before you write your landing page.

4. Product Pillars

You picked all three as differentiators. That's a legitimate 12-18 month vision — but sequence them, don't build them in parallel:

Pillar	What it means	When
Effortless capture	Paste any link, any format, zero friction	v1
Resurfacing	Reminds you to revisit saved items	v1 (user-set reminders)
Semantic search & knowledge graph	Find and visually connect saved items by meaning, not just keywords	v2+

5. Competitive Landscape

This space is real but not saturated the way it looks:

- **Raindrop.io** — the strongest free-tier bookmark manager, collections + tags, no real AI or knowledge graph.

- **Readwise Reader** — best-in-class for articles/highlights + spaced-repetition resurfacing, but weak on non-text formats (tweets, arbitrary links) and has no knowledge graph.
- **Notion Web Clipper** — captures into Notion, but organizing is entirely manual; no AI tagging or resurfacing.
- **Mem** — closest in spirit (AI-organized personal knowledge base) but is note-taking-first, not save-first, and capture friction is higher.
- **Pocket** — shut down by Mozilla in 2025, which quietly removed the most mainstream "just save it" option from the market and pushed its users toward Raindrop/Readwise.

Nobody currently owns "one-click save any format + AI tags + custom resurfacing" as a combined default. That combination is your actual wedge — not any single pillar alone.

6. MVP Scope (v1) — Web app only, no extension yet

In scope:

- Paste-a-link capture (article, tweet, image, YouTube video, PDF link) via a web app — no browser extension required
- Automatic metadata fetch per type (Open Graph for articles, oEmbed for tweets, YouTube API for videos, text extraction for PDFs)
- Automatic AI tag suggestions, editable/overridable by the user
- Manual collections (folders) in addition to tags
- Per-item custom reminders ("remind me about this in 3 days / 2 weeks / custom date")
- Basic keyword search across saved items
- A simple feed/list view of saved items, filterable by tag/collection

Explicitly out of scope for v1 (this is the discipline that keeps the launch shippable):

- Browser extension
- Mobile app / share-sheet
- Knowledge graph visualization
- Semantic (embedding-based) search
- Highlight system
- Automatic (non-user-set) resurfacing schedules

7. Roadmap After v1

v2 — Capture expansion + smarter retrieval

- Browser extension (this is where "save anything" stops requiring a copy-paste step)
- Semantic search (embeddings) alongside keyword search
- Highlight/annotation system for articles and PDFs

v3 — The knowledge graph pillar

- Topic clustering across saved items
- Graph visualization of related items
- "Related items" surfaced automatically when viewing any saved item
- Mobile app / share-sheet capture

Resist pulling v2/v3 items into v1 scope — this is the single biggest risk to actually shipping (see Risks below).

8. Core User Flows

Save flow: paste URL → system detects type → fetches metadata + content → generates AI tag suggestions → user confirms/edits tags, optionally sets a reminder or assigns a collection → item stored.

Resurfacing flow: reminder date arrives → user gets a notification (email at minimum for v1; push later) → clicking it opens the item directly.

Search flow: user searches → v1 does keyword match on title/tags/extracted text → v2 adds "find items similar in meaning" via embeddings.

9. Data Model (core entities)

- **User** — auth, plan tier
- **Item** — type (article/tweet/image/video/pdf/link), source URL, extracted content/text, metadata (title, thumbnail, author), created_at
- **Tag** — name, AI-generated flag, linked to Items (many-to-many)
- **Collection** — user-created folder, linked to Items (many-to-many)
- **Reminder** — linked to one Item, trigger date, sent flag
- **Embedding** — vector representation of an Item's content (v2+)

10. Technical Architecture

Matches the stack you already sketched:

Frontend: Next.js (React) — web app first, no extension/mobile build needed for v1.

Backend: Node or Python service handling:

- Ingestion API (receives pasted URL, kicks off processing)
- Metadata/content extraction per source type
- AI tagging calls (LLM API)
- Reminder scheduling

Queue workers: essential from day one, not a "later" optimization — link fetching, PDF parsing, and AI tagging are all slow/unreliable enough that they can't block the save action. User pastes a link → item appears instantly as "processing" → worker fills in metadata/tags asynchronously.

Storage:

- Postgres for structured data (Users, Items, Tags, Collections, Reminders)
- Object storage (S3-compatible) for saved images/PDFs
- Vector DB — can be deferred to v2 (Postgres + pgvector is enough to start when you get there; no need for a dedicated vector DB at v1 scale)

Scheduled job: a daily/hourly cron that checks due reminders and sends notifications.

11. Business Model — Freemium

Free tier (needs to be genuinely useful, or freemium doesn't convert):

- Item cap (e.g. 100–150 saved items)
- Manual reminders, basic search, tagging

Paid tier:

- Unlimited items
- Semantic search + knowledge graph (once built)
- Browser extension + mobile capture
- Priority AI tagging / higher usage limits

Comparable tools price the paid tier in the \$6–10/month range — reasonable anchor, not a fixed recommendation; validate against your actual AI + storage costs once you have usage data, since embedding and LLM calls are your main variable cost per user.

12. Key Risks

- **Scope creep** — the biggest one. "All three pillars" is the vision; shipping v1 with only capture + manual reminders is what gets you real users to learn from.
- **Extraction reliability** — paywalled articles, dynamic JS-rendered pages, and rate-limited APIs (Twitter/X, YouTube) will break naive scraping. Budget real time for this; it's the unglamorous 40% of the build.
- **AI cost per item** — tagging every saved item with an LLM call adds up at scale; watch this before it becomes a margin problem.
- **Reminder fatigue** — if resurfacing feels like nagging rather than useful, users churn. Worth tracking reminder-click-through rate from day one.

13. Suggested Success Metrics (v1)

- Items saved per active user per week
- % of saved items that get reopened at all (the core retrieval problem you're solving)
- Reminder click-through rate
- Week-4 retention

14. Naming Ideas

Save-anything + resurfacing tools tend to lean on memory/mind language: **Recall**, **Resurface**, **Mindbase**, **Loopback**, **Keepsake**, **Reflex** (as in — it reflexively brings things back to you). Worth checking trademark/domain availability before getting attached to one.